# AI WEATHERWISE

*Intelligent Weather Forecasting and AI-Powered Weather Insights*

---

# PHASE 5 – PROJECT DEVELOPMENT

Project Setup, Backend Development and Database Development

**Project Type**
AI-powered weather forecasting and recommendation REST API / backend

**Technology Stack**
Node.js · Express.js · MongoDB Atlas · Mongoose · JWT · bcrypt.js · Google Gemini AI · OpenWeatherMap · REST APIs · Thunder Client / Postman

**Backend Port**
5001

**Document Date**
September 29, 2026

**Status**
Project Development Phase

*BSc Computer Science Project*

# 1. Project Setup

**This phase documents how AI WeatherWise is set up and developed. The content is based on the AI WeatherWise project documentation. Where that documentation does not specify an implementation detail, the detail is described generically or marked as "to be confirmed" instead of being assumed.**

## 1.1 Node.js Environment

AI WeatherWise runs on Node.js (LTS version; the project documentation lists Node.js v16 or later), which executes the JavaScript backend on the server. Because the application is a headless REST API, there is no browser-side interface; all features are reached through HTTP requests that return JSON. Before development starts, the installed versions can be checked with the commands node -v and npm -v.

## 1.2 npm Package Management

npm creates and manages the project. Running the initialization command generates package.json, which records the project details, the dependency list and the run scripts. Dependencies are then installed with npm install and are stored in the node_modules folder. The node_modules folder and the .env file are excluded from Git through .gitignore, so that generated files and secrets are never pushed to the repository.

```
npm init                    # create package.json
npm install                 # install all dependencies listed in
package.json
```

## 1.3 Required npm Dependencies

The packages below are those identified in the AI WeatherWise project documentation.

*Table 1: Required npm Dependencies*

| Package | Purpose |
|---|---|
| **express** | Web framework for the server, routes and middleware. |
| **mongoose** | Connects to MongoDB Atlas and defines the User and Location schemas and models. |

| Package | Purpose |
|---|---|
| **jsonwebtoken** | Creates and verifies JWT tokens for authentication and protected routes. |
| **bcryptjs** | Hashes passwords before storage and compares them at login (bcrypt.js). |
| **cors** | Enables cross-origin requests to the API. |
| **dotenv** | Loads environment variables (PORT, MONGO_URI, JWT_SECRET, GEMINI_API_KEY) from the .env file. |
| **@google/genai** | Google Gemini SDK used by the Gemini AI service to generate weather summaries and recommendations. |
| **multer** | Middleware dependency installed for handling multipart/form-data; none of the documented WeatherWise features depends on it. |
| **nodemon (development dependency)** | Restarts the server automatically when files change during development. |
| **Weather API integration** | The weather service communicates with the OpenWeatherMap API. The project documentation does not name a separate client package for this; confirm against package.json. |

## 1.4 Environment Variables

Environment variables keep configuration and secrets outside the source code. They are stored in a .env file, loaded with dotenv, and read in code through process.env. The .env file is listed in .gitignore so it is never pushed to GitHub. Real secret values are never written in documentation or source code; only placeholders are shown below.

*Table 2: Environment Variables*

| Variable | Example (placeholder) | Purpose |
|---|---|---|
| **PORT** | `5001` | Port on which the AI WeatherWise server listens. |
| **MONGO_URI** | `<your-mongodb-atlas-connection-string>` | Connection string to the MongoDB Atlas cluster. It contains database credentials. |
| **JWT_SECRET** | `<long-random-secret>` | Key used to sign and verify JWT tokens. |
| **GEMINI_API_KEY** | `<your-gemini-api-key>` | Authorizes requests to the Google Gemini API. |

| Variable | Example (placeholder) | Purpose |
|---|---|---|
| **Weather API key (variable name to be confirmed)** | `<your-weather-api-key>` | Authorizes requests to OpenWeatherMap. The source documents only that an external weather API key may be absent, in which case fallback (mock) data is returned. |

Why environment variables are used for sensitive values:

- **MongoDB connection string:** it contains the database username and password. If it leaks, anyone can read or change the stored user and location data.
- **JWT secret:** it signs every token. Anyone who knows it can create valid tokens and impersonate any user.
- **Gemini and weather API keys:** they are tied to the owner's accounts and usage quotas. Leaked keys can be misused and consume that quota.

Keeping these values out of the code also lets the same code run with different values on different machines, and a compromised value can be replaced without editing the code.

## 1.5 Server Configuration

server.js is the application entry point. It loads the environment variables, connects to MongoDB Atlas and starts the Express application, which listens on the port set in the PORT variable (5001). The Express application itself is configured separately in app.js.

The project documentation does not describe what happens when the database connection fails. The expected practice for this design is to log the error and stop the process instead of accepting requests without a database; this behaviour should be confirmed against the implementation.

## 1.6 MongoDB Connection Configuration

The connection logic is placed in config/db.js, which establishes the MongoDB connection through Mongoose using the connection string from MONGO_URI. Before the connection can work, the following are prepared in MongoDB Atlas:

- A MongoDB Atlas cluster is created to host the AI WeatherWise database.
- A database user is added, whose username and password appear inside MONGO_URI.
- The developer's IP address is allowed in the Atlas network access settings.

The connection string is stored in MONGO_URI and passed to Mongoose's connect() function. A successful connection allows the User and Location collections to be used; a failed connection is reported as an error (see Section 1.5 for the behaviour that remains to be confirmed).

## 1.7 API Testing Setup

Once the server is running, the API is tested by sending requests from Thunder Client or Postman to the local server address on port 5001 (for example, http://localhost:5001). The project's API testing evidence in the source documentation was captured with Postman, and a Postman collection file (postman_collection.json) is part of the project folder structure.

*Table 3: API Testing Setup for AI WeatherWise*

| Feature | Endpoint | JWT Required | Documentation Status |
|---|---|---|---|
| **Register API** | `POST /api/auth/register` | No | Tested: 201 Created with user object and JWT. |
| **Login API** | `POST /api/auth/login` | No | Tested: 200 OK with user object and JWT. |
| **Weather API** | `GET /api/weather/:city` | No (public) | Tested: 200 OK with city, temperature, humidity, windSpeed, condition; isMock flag on fallback data. |
| **AI Weather Recommendation API** | `POST /api/ai/weather-recommendation` | Not confirmed in source | Tested: 200 OK with a Gemini-generated recommendation. |
| **AI Weather Summary** | `Not specified in source` | To be confirmed | Described as part of the AI service; no separate tested endpoint shown. |
| **Favorite Location APIs (create, view, update, delete)** | `Not specified in source` | Yes | Documented through architecture and data model; route paths not shown. |

Protected APIs, such as favorite location management, require a valid JWT sent as a Bearer token in the Authorization header. The token is obtained from the login response, and requests without a valid token are rejected by the authentication middleware. Public routes, such as the current-weather lookup, can be tested without a token.

## 1.8 Setup Summary Table

*Table 4: Setup Summary*

| Setup Component | Technology / Configuration | Purpose |
|---|---|---|
| **Runtime** | Node.js (LTS) | Runs the JavaScript backend on the server. |
| **Package Management** | npm, package.json | Installs dependencies and defines run scripts. |
| **Web Framework** | Express.js | Handles HTTP requests, routing and middleware. |
| **Folder Structure** | src/ with config, controllers, middleware, models, routes, services, app.js and server.js | Separates responsibilities for a maintainable codebase. |
| **Environment Variables** | .env with dotenv | Keeps the MongoDB string, JWT secret and API keys out of the code. |
| **Server Configuration** | server.js, PORT = 5001 | Loads configuration, connects to the database and starts listening. |
| **Database** | Mongoose + MongoDB Atlas (config/db.js) | Connects the application to the cloud database. |
| **AI Integration** | Google Gemini AI via @google/genai | Generates weather summaries and recommendations. |
| **Weather API Integration** | OpenWeatherMap through the Weather Service | Supplies live current-weather data, with fallback (mock) data when the key is not configured. |
| **Development Execution** | nodemon (run script defined in package.json) | Runs the server with automatic restart. |
| **API Testing Tool** | Thunder Client / Postman | Sends requests and checks JSON responses. |

# 2. Backend Development

## 2.1 Express Application

The Express application (app.js) is the core of the backend. It configures the middleware, mounts the route files and connects each request path to its controller. It is kept separate from server.js, so that the application setup is independent of starting the server and connecting to the database. Errors from any layer are passed to the centralized error handler, which returns a consistent JSON response.

## 2.2 Middleware

*Table 5: Middleware in AI WeatherWise*

| Middleware | Role |
|---|---|
| **JSON middleware (express.json())** | Parses JSON request bodies so controllers can read the submitted data. |
| **CORS (cors())** | Manages CORS headers so that cross-origin requests to the API are handled. |
| **Request sanitization** | Sanitizes incoming request data as part of request processing. The specific library or method is not named in the source. |
| **Authentication middleware** | Reads the Bearer token from the request headers, verifies the JWT, identifies the authenticated user and protects private endpoints. Requests with an invalid or missing token are rejected as unauthorized; the exact status code should be confirmed against the implementation. |
| **Centralized error handler** | Receives errors from the application layers and returns a consistent JSON error response. |

No request-logging middleware is documented for AI WeatherWise, so none is described here.

## 2.3 Routes, Controllers and Services

AI WeatherWise follows a layered MVC-style architecture. Because the application is a headless REST API, the “view” layer is the routing layer that maps request paths to controllers.

*Routes → Middleware → Controllers → Services → Models → MongoDB Atlas*

- **Routes** connect an HTTP method and URL to a controller function and attach the authentication middleware to protected endpoints. They are grouped into authentication

(authRoutes), weather (weatherRoutes), favorite location (locationRoutes) and AI (aiRoutes) routes.

- **Controllers** handle the request and response: they receive the request, apply input constraints, delegate the work to services or models and return the standard JSON response. The project has Auth, Weather, Location and AI controllers.
- **Services** hold logic that communicates with external systems. The Weather Service calls the OpenWeatherMap API, and the Gemini AI Service calls Google Gemini AI through the @google/genai SDK. In the documented design, password hashing is performed by the Auth controller with bcrypt, and favorite location operations are performed by the Location controller through its model.
- **Models** define the schemas for the User and Location collections and communicate with MongoDB Atlas through Mongoose.

The weather and AI integrations follow these paths:

*Controller → Weather Service → OpenWeatherMap / Weather API*

*Controller → Gemini AI Service → Google Gemini API → AI Weather Summary / Recommendation*

The documented backend folder structure is shown below.

```
WEATHER/
|-- src/
|   |-- config/         db.js
|   |-- controllers/    aiController.js, authController.js,
|   |                   locationController.js, weatherController.js
|   |-- middleware/     authMiddleware.js
|   |-- models/         Location.js, User.js
|   |-- routes/         aiRoutes.js, authRoutes.js,
|   |                   locationRoutes.js, weatherRoutes.js
|   |-- services/       (weather / gemini service modules)
|   |-- app.js
|   `-- server.js
|-- .env
|-- .gitignore
|-- package.json
|-- package-lock.json
|-- postman_collection.json
`-- README.md
```

## 2.4 Modular Structure and Request Lifecycle

A request enters through the Express application, passes the global middleware, matches a route, passes through the authentication middleware if the route is protected, reaches a controller, and is handled by a service or model that uses MongoDB Atlas, the weather API or Google Gemini AI. Errors at any step go to the centralized error handler. Because each layer only talks to the next one, a change in one place, such as changing the AI prompt, does not affect the rest of the code.

*Client / Thunder Client → Express Application → Middleware → Route → Authentication Middleware (if protected) → Controller → Service → MongoDB / Weather API / Gemini AI → JSON Response*

**A. Weather data request flow.** The client sends a request with a city name. This route is public, so no token is needed. The Weather Controller passes the city to the Weather Service, which requests current weather from OpenWeatherMap and returns temperature, humidity, wind speed and condition. If the external weather API key is not configured, the service returns a structured fallback response flagged with isMock, so the endpoint keeps working. The controller sends the result as JSON.

*Client → Weather Route (public) → Weather Controller → Weather Service → OpenWeatherMap (or fallback data) → JSON Response*

**B. AI weather insight request flow.** The client submits weather details (city, temperature, humidity and condition). The AI Controller forwards them to the Gemini AI Service, which builds the request and calls Google Gemini AI. The generated natural-language recommendation or summary is returned to the client as JSON. Because the AI features work from weather details, the weather information is obtained first.

*Client → AI Route → AI Controller → Gemini AI Service → Google Gemini AI → Recommendation / Summary → JSON Response*

**C. Favorite location request flow.** The client sends a request with a valid Bearer JWT. The authentication middleware verifies the token, and the Location Controller creates, reads, updates or deletes favorite locations through the Location model in MongoDB Atlas. New locations store the authenticated user's ObjectId in the user field, and update and delete operations include an ownership check.

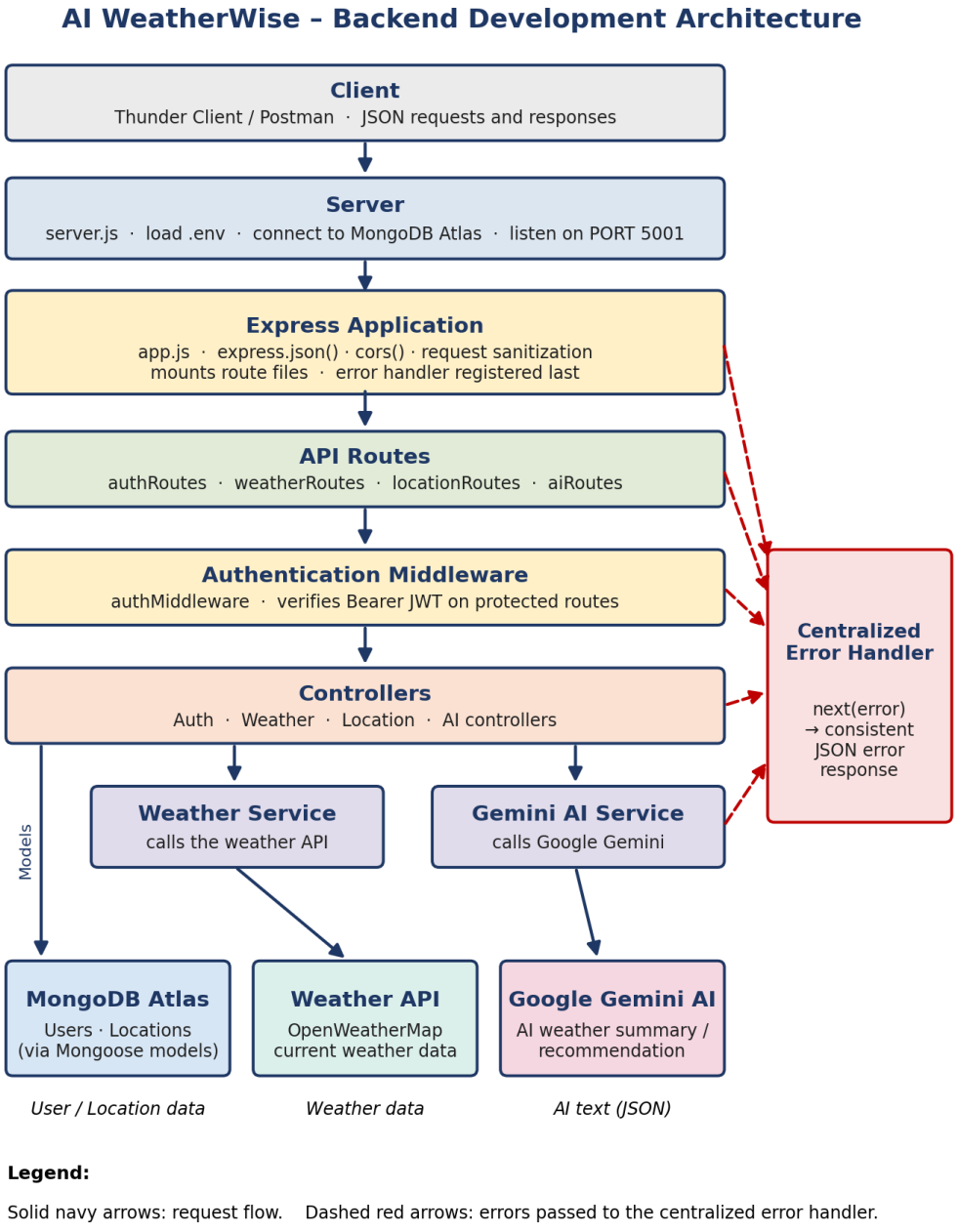



*Figure 1: AI WeatherWise – Backend Development Architecture*

*Table 6: Architecture Components*

| Component | Responsibility |
|---|---|
| **Client / Thunder Client / Postman** | Sends HTTP requests and receives JSON responses. |
| **Server (server.js)** | Loads environment variables, connects to MongoDB Atlas and starts Express on port 5001. |

| Component | Responsibility |
|---|---|
| **Express Application (app.js)** | Configures global middleware and mounts the route files. |
| **API Routes** | Map methods and paths to controllers for authentication, weather, favorite locations and AI. |
| **Authentication Middleware** | Verifies the Bearer JWT before private routes are used. |
| **Controllers** | Handle requests, call services or models and return JSON. |
| **Weather Service** | Communicates with OpenWeatherMap and provides fallback data when the key is not configured. |
| **Gemini AI Service** | Communicates with Google Gemini AI to generate summaries and recommendations. |
| **Models and MongoDB Atlas** | Mongoose models store the User and Location data in the cloud database. |
| **Centralized Error Handler** | Converts errors from any layer into a consistent JSON error response. |

# 3. Database Development

## 3.1 MongoDB Atlas and Mongoose

MongoDB Atlas is the cloud database that stores all application data in two collections: Users and Locations. Mongoose is the object modeling library that connects Node.js to MongoDB. Each collection is defined by a Mongoose schema, which sets the fields, their data types and their validation rules, and Mongoose creates the model used to read and write data. Both schemas use the timestamps option, so createdAt and updatedAt are added and maintained automatically.

## 3.2 User Model

*Table 7: User Model*

| Field | Type | Rules / Description |
|---|---|---|
| `_id` | ObjectId | Primary key, generated automatically by MongoDB. |
| `name` | String | Required. The user's full name. |
| `email` | String | Required and unique, so two accounts cannot share an email address. Checked against a standard email pattern and stored in lowercase. Used to log in. |
| `password` | String | Required. Minimum 6 characters before hashing. Stored only as a bcrypt hash, never as plain text. |
| `createdAt` | Date | Set automatically when the user is created (Mongoose timestamps). |
| `updatedAt` | Date | Updated automatically on every change (Mongoose timestamps). |

The project's implemented User schema contains no role field. A separate section of the project documentation describes Admin and Registered User responsibilities, but these are not represented in the schema, so no role field is documented here.

## 3.3 Location Model

*Table 8: Location (Favorite Location) Model*

| Field | Type | Rules / Description |
|---|---|---|
| `_id` | ObjectId | Primary key, generated automatically by MongoDB. |

| Field | Type | Rules / Description |
|---|---|---|
| `user` | ObjectId (ref: User) | Required. Reference to the User who owns the favorite location. |
| `city` | String | Required. Name of the saved favorite city. |
| `country` | String | Required. Country associated with the saved city. |
| `createdAt` | Date | Set automatically when the record is created (Mongoose timestamps). |
| `updatedAt` | Date | Updated automatically on every change (Mongoose timestamps). |

A compound unique index on user, city and country ensures that a user cannot save the same city and country combination twice.

## 3.4 Validation, Uniqueness and Password Storage

- **Required fields:** name, email and password are required for a user, and user, city and country are required for a favorite location. Mongoose rejects a document if a required field is missing.
- **Email validation and uniqueness:** the email is checked against a standard email pattern, stored in lowercase, and protected by a unique index enforced by MongoDB, so duplicate registrations fail.
- **Password rules and storage:** the password must be at least 6 characters long and is hashed with bcrypt.js before saving. At login, the submitted password is compared with the stored hash using bcrypt.compare; the original password cannot be recovered from the hash.
- **Duplicate favorite prevention:** the compound unique index on user, city and country blocks duplicate favorite locations for the same user.
- **Request sanitization:** incoming request data is sanitized as part of request processing.
- **Consistent error responses:** validation and other errors are handled by the centralized error handler, so the API reports errors in a consistent way. The source does not document specific failure status codes, so none are stated here.

## 3.5 User–Location Relationship

The relationship is one-to-many: one User can have many favorite Locations, and each Location belongs to exactly one User.

*ONE USER → MANY FAVORITE LOCATIONS*

The Location model's user field stores the owning User's _id with ref: User. When a favorite location is created, this field is filled with the authenticated user's ObjectId. Location operations are scoped to that user, so an authenticated user manages only their own favorite locations. Mongoose populate() can be used to replace the reference with the user document when needed.

## 3.6 AI WeatherWise Database Structure

The database structure below shows the two collections and the relationship between them.

```
MongoDB Atlas
|
|-- Users
|      |-- _id
|      |-- name
|      |-- email
|      |-- password
|      |-- createdAt
|      `-- updatedAt
|
`-- Locations
       |-- _id
       |-- user        (reference to Users._id)
       |-- city
       |-- country
       |-- createdAt
       `-- updatedAt

Relationship:   User 1 ---------- N Location
```

The Location.user field references the corresponding User document. Figure 2 shows the same structure as a diagram.

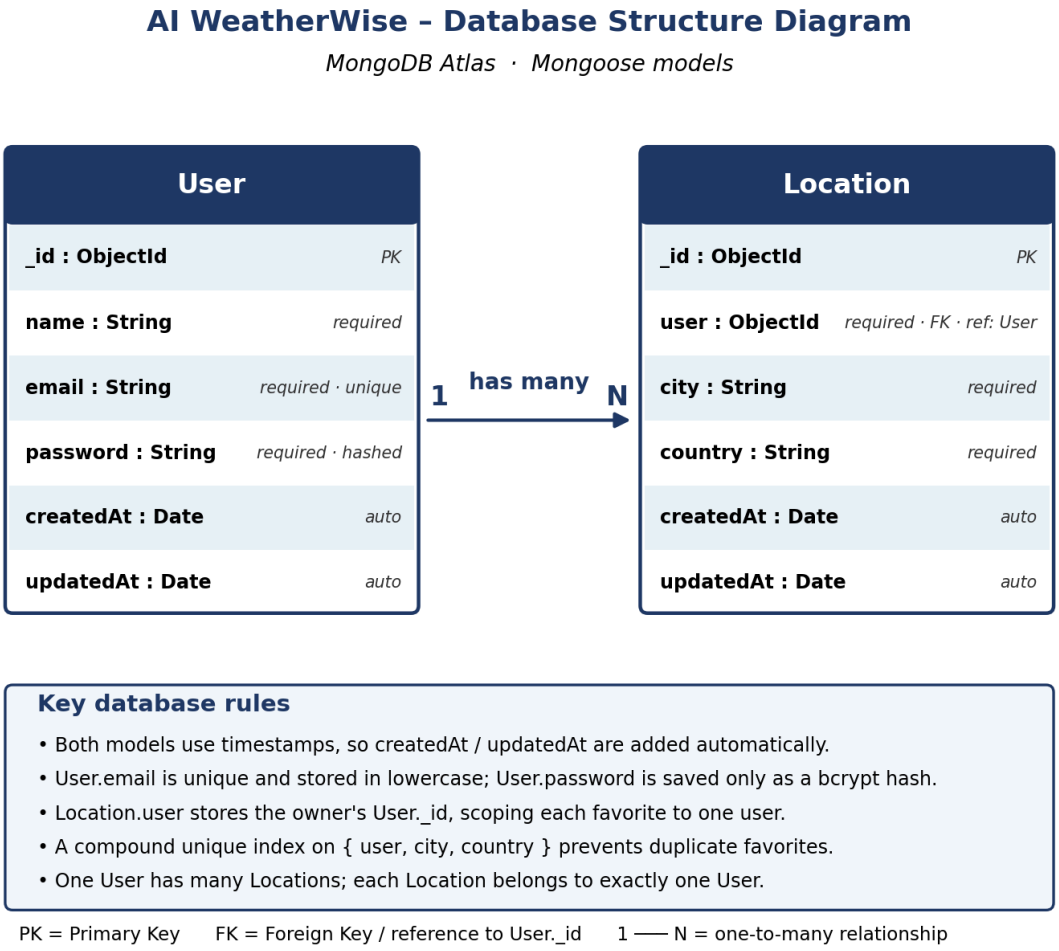



*Figure 2: AI WeatherWise – Database Structure Diagram*

The project's entity-relationship diagram also documents broader conceptual entities, such as weather history, AI insight records, alerts, subscriptions, API usage logs and AI models. These are not implemented as Mongoose schemas in the current codebase, so they are not part of this database structure.